

Fintech Platform API Design

Every Endpoint, Every Route, and the Reasoning Behind Each One

A complete REST API reference for accounts, cards, transactions, loans, and investments

Companion document to the Fintech Platform Data Model schema

June 2026

Contents

1. How to Read This Document: REST Conventions Used Throughout
2. Authentication & Session Endpoints
3. User & KYC Endpoints
4. Account Endpoints
5. Card Endpoints
6. Transaction & Ledger Endpoints
7. Loan Endpoints
8. Invoice Endpoints
9. Investment & Brokerage Endpoints
10. Webhook Endpoints (Incoming, From External Providers)
11. Admin & Compliance Endpoints
12. Full Endpoint Index
13. Error Format Reference
14. Versioning, Rate Limiting & Deprecation Strategy
15. Suggested Build Order & Glossary

1. How to Read This Document: REST Conventions Used Throughout

This document lists every endpoint your fintech platform needs, grouped by resource, in the same order as the entities in the Fintech Platform Data Model schema document. Each endpoint is shown with its HTTP method, path, a one-line summary, who is allowed to call it, request/response shapes, and the specific reasoning an interviewer or a future you debugging a bug at 2am would want to see spelled out.

Before the endpoint lists, this section fixes a handful of conventions once so they don't need repeating five hundred times. Every endpoint in this document follows these rules unless explicitly stated otherwise.

1.1 Base URL and versioning

All endpoints are mounted under a versioned base path:

```
https://api.yourplatform.com/v1/...
```

URI-based versioning (the `/v1/` prefix) is the most practical choice for a learning project and for most production APIs: it is the most visible to API consumers, the easiest to route at the infrastructure level (a load balancer or API gateway can send `/v1/*` and `/v2/*` to entirely different backend deployments), and the easiest to debug by hand with a tool like curl. The alternative -- a version header like Stripe's date-based versioning -- gives finer control over per-field transformations but is meaningfully more complex to implement correctly, and is the kind of thing to graduate to once v1 is stable, not to start with.

The rule that matters more than which scheme you pick: once `/v1/` is published and a client depends on it, it is never broken. New, non-breaking fields can be added to responses at any time. A breaking change (removing a field, changing a field's type, changing required-ness) ships under `/v2`, with `/v1` kept running in parallel until a deprecation window closes.

1.2 Authentication

Every endpoint except registration, login, and public reference-data lookups requires a bearer token in the Authorization header:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

This document marks each endpoint's required auth level using the labels below, shown in the Auth column of the quick-reference table in Section 12 and noted under each detailed endpoint.

Auth levels used throughout this document

Label	Meaning
Public	No token required. Registration, login, public reference data (currency list, card networks).
User	A valid user-scoped bearer token is required. The endpoint only operates on

Label	Meaning
	resources owned by that authenticated user -- enforced server-side, never trusted from a client-supplied <code>user_id</code> .
User + 2FA	A valid token is required, and the specific request must additionally include a recent second-factor confirmation (Section 2.4) -- used for high-risk actions like adding a new card or raising a withdrawal limit.
Admin	Restricted to internal staff/back-office roles, not exposed to the mobile or web client used by ordinary users.
System	Called only by your own backend's internal services or scheduled jobs (e.g. the brokerage-sync worker), never by an end-user-facing client, and typically not exposed past your internal network at all.
Webhook (signed)	Called by an external provider (a payment processor, a brokerage). Authenticated by verifying a cryptographic signature in a request header, not a bearer token -- see Section 10.1.

1.3 Idempotency on every mutating request

Every POST and PATCH endpoint that has a real-world financial side effect (creating a transaction, disbursing a loan, charging a card) requires an Idempotency-Key header, following the same convention Stripe popularized and that the HTTP working group has since standardized as a named header:

```
POST /v1/transactions
Idempotency-Key: 8a6f3c2e-9b1d-4f5a-8c3e-7d2a1b9f0c4e
Content-Type: application/json
```

The client generates this key once, at the moment the user expresses intent (the instant they tap 'Send' or 'Pay'), and resends the exact same key on every retry of that same logical action. Server-side behavior is standardized:

- New key: process the request normally, store the key together with the resulting response body and status code.
- Repeated key, identical payload: skip processing, return the originally stored response unchanged -- the caller cannot tell the difference between the first call and a replay.
- Repeated key, different payload: reject with 422 Unprocessable Content. A key must mean one specific operation; silently applying it to a different payload would hide a client bug.
- Repeated key, original request still mid-flight: reject the second concurrent attempt with 409 Conflict rather than letting both proceed -- this closes the race condition where a slow first request and an eager retry both reach the database before either has finished.
- Missing key on an endpoint that requires one: reject with 400 Bad Request before touching any business logic.

Keys are retained for a fixed deduplication window (24 hours is the common industry default) and can be purged after -- there's no need to remember a key forever, only long enough to outlast realistic client retry behavior.

1.4 Request and response format

- All request and response bodies are JSON, with Content-Type: application/json.
- All monetary amounts in requests and responses are integers in minor units (kobo, cents) -- never decimal/float -- consistent with the schema document's amount_minor convention. A response showing 500000 for a NGN amount means ₦5,000.00; the client divides by 100 (or by currency.minor_unit_decimals) only for display.
- All timestamps are ISO 8601 in UTC, e.g. 2026-06-27T14:32:00Z. Client-local display formatting happens on the client, never the server.
- All resource identifiers are UUIDs, matching the schema's primary key convention.
- Field names use snake_case in JSON bodies, matching the database column names directly -- this keeps the mapping between API payloads and schema fields obvious while you're learning, though many teams choose camelCase for JSON by convention; either is fine as long as it's consistent.

1.5 Pagination

Any endpoint that lists more than a handful of records (transaction history, audit logs, investment holdings) uses cursor-based pagination rather than page-number/offset pagination. This matters specifically for financial data: offset pagination (?page=3&limit=20) forces the database to count and skip rows on every request, and new transactions arriving between two page requests cause rows to be skipped or duplicated in the client's view -- a correctness bug, not just a performance one, for data your user is using to reconcile their own spending.

```
GET /v1/accounts/{account_id}/transactions?limit=20&after=eyJpZCI6IjQ0N...
```

Response:

```
{
  "data": [ { "transaction_id": "...", ... }, ... ],
  "next_cursor": "eyJpZCI6IjQ0OCJ9",
  "has_more": true
}
```

The cursor (next_cursor) is an opaque, server-generated token -- typically a base64-encoded pointer to the last record's id and created_at. Clients never construct or decode it; they just pass whatever the server gave them back as the after parameter on the next request. A null next_cursor signals the final page.

1.6 Error format

Every error response follows a single, consistent shape across every endpoint in this document, based on RFC 9457 (Problem Details for HTTP APIs) -- a published IETF standard for exactly this purpose, rather than an in-house convention each team reinvents slightly differently.

```
HTTP/1.1 422 Unprocessable Content
Content-Type: application/problem+json
```

```
{
  "type": "https://api.yourplatform.com/errors/insufficient-funds",
  "title": "Insufficient funds",

```

```

"status": 422,
"detail": "Account balance (450000) is less than the requested debit (500000).",
"instance": "/v1/transactions/8a6f3c2e-...",
"request_id": "req_3f9a2b1c"
}

```

The full reference for status codes and the standard error 'type' values used across this API is in Section 13 -- it's worth reading that section once up front, since every endpoint below assumes it.

1.7 Rate limiting

Every response includes standard rate-limit headers so a client can self-throttle before hitting a hard block:

```

X-RateLimit-Limit: 100
X-RateLimit-Remaining: 87
X-RateLimit-Reset: 1719500000

```

When a client exceeds its limit, the server responds 429 Too Many Requests with a Retry-After header indicating how many seconds to wait. Authenticated endpoints are typically rate-limited per user_id; public/unauthenticated endpoints (registration, login) are rate-limited per IP address specifically to slow down credential-stuffing and brute-force attempts.

1.8 How each endpoint entry is laid out

Each endpoint below is shown the same way, so you can scan the whole document quickly once you're used to the format:

Reading an endpoint entry

Element	Meaning
Colored method badge	GET (green), POST (blue), PATCH/PUT (amber), DELETE (red) -- color is consistent throughout so you can visually scan a page for 'what mutates vs. what only reads'.
Path	The full route, with {placeholder} segments for path parameters.
One-line summary	What the endpoint does, in plain language.
Auth	Which auth level from Section 1.2 is required.
Idempotent?	Whether an Idempotency-Key header is required, optional, or not applicable (GET/DELETE by nature don't need one -- see the callout in Section 6).
Request / Response	Representative JSON bodies, trimmed to the fields that matter for understanding the shape -- not always every field from the schema.
Notes	The design reasoning -- why the endpoint is shaped this way, what schema entity it maps to, and any gotchas.

2. Authentication & Session Endpoints

These endpoints do not map to a single schema table the way later sections do -- they sit in front of USER, handling identity proof and session issuance. Getting this section right matters more than its size suggests: almost every security incident in a fintech app traces back to a gap here, not in the ledger logic.

2.1 Register

POST	/v1/auth/register	<i>Create a new user account.</i>
Auth: Public		
Idempotent?: Recommended (Idempotency-Key) -- prevents a double-tapped signup form from creating two users with the same email before the unique constraint even gets a chance to reject the second one cleanly.		
Request body:		
<pre>{ "email": "seyi@example.com", "password": "a-strong-passphrase", "first_name": "Seyi", "last_name": "Adebayo", "date_of_birth": "2002-04-11", "country_code": "NG", "default_currency_code": "NGN" }</pre>		
Response 201 Created:		
<pre>{ "user_id": "8a6f3c2e-9b1d-4f5a-8c3e-7d2a1b9f0c4e", "email": "seyi@example.com", "kyc_status": "unverified", "created_at": "2026-06-27T14:32:00Z" }</pre>		

Notes: the response never echoes back the password, even hashed -- there is no legitimate client need for it, and returning any form of it just widens the blast radius if a response ever gets logged somewhere it should not. Password hashing happens server-side before the row is written; the plaintext password is never persisted, not even momentarily in a way that would show up in a slow-query log. Account creation does not automatically grant the ability to move money -- that only happens after KYC verification, covered in Section 3.

2.2 Login

POST	/v1/auth/login	<i>Exchange credentials for an access token and refresh token.</i>
Auth: Public (but IP-rate-limited aggressively -- see Section 1.7)		
Idempotent?: Not applicable -- logins are naturally safe to repeat; each one is a new session, not a duplicate side effect.		
Request body:		
<pre>{ "email": "seyi@example.com", "password": "a-strong-passphrase" }</pre>		
Response 200 OK:		
<pre>{ "access_token": "eyJhbGciOiJIUzI1NiIs... ", "refresh_token": "8f14e45fceeal67a5a36...", }</pre>		

```

"expires_in": 900,
"two_factor_required": false
}

```

Notes: access_token is short-lived (15 minutes is a common default -- expires_in is in seconds) and is what gets sent as the Bearer token on every subsequent request. refresh_token is longer-lived and stored more carefully by the client because it is what gets exchanged for a new access_token once the short one expires, without forcing the user to type their password again. If two_factor_required is true, the access_token returned is deliberately scoped down -- it can only call the 2FA verify endpoint below, nothing else, until the second factor is confirmed.

2.3 Refresh access token

POST	/v1/auth/refresh	<i>Exchange a valid refresh token for a new access token.</i>
Auth: Public (the refresh_token itself is the credential)		
Request body: { "refresh_token": "8f14e45fcee167a5a36..." }		
Response 200 OK: { "access_token": "eyJ...", "expires_in": 900 }		

Notes: this is the endpoint a mobile app calls silently in the background every fifteen minutes or so, so the user never has to re-login during a session. If the refresh token itself has expired or was revoked (Section 2.6), this returns 401 Unauthorized and the client falls back to the full login screen.

2.4 Two-factor authentication: enable, verify, disable

POST	/v1/auth/2fa/enable	<i>Begin 2FA setup -- returns a TOTP secret and QR code.</i>
Auth: User		
POST	/v1/auth/2fa/verify	<i>Confirm a TOTP code, either to finish enabling 2FA or to satisfy a login challenge.</i>
Auth: User (possibly a scoped-down token, per Section 2.2)		
POST /v1/auth/2fa/verify { "code": "482913" }		
Response 200 OK: { "access_token": "eyJ... (full-scope now)", "expires_in": 900 }		
DELETE	/v1/auth/2fa	<i>Disable 2FA on the account.</i>
Auth: User + 2FA		

Notes: disabling 2FA is itself gated behind a fresh 2FA confirmation, not just a normal bearer token -- otherwise stealing a single access token would be enough to both disable the user's second factor and then proceed unguarded, defeating the entire point of having one. This is the canonical example of why the User + 2FA auth level exists as distinct from plain User in Section 1.2's table.

2.5 Password reset

POST	/v1/auth/password/forgot	<i>Request a password reset link or code via email.</i>
Auth: Public (rate-limited per email address, not just per IP, to stop someone enumerating which emails are registered)		
POST	/v1/auth/password/reset	<i>Complete a password reset using the emailed token.</i>
Auth: Public (the reset token itself is the credential)		

Notes: the forgot-password endpoint always returns the same 200 OK response regardless of whether the email exists in the system. Returning a different response for an unknown email versus a known one is a classic account-enumeration leak -- it lets an attacker silently build a list of valid emails on your platform by trying many addresses and watching which ones differ.

2.6 Logout and session management

POST	/v1/auth/logout	<i>Revoke the current refresh token.</i>
Auth: User		
GET	/v1/auth/sessions	<i>List active sessions/devices for the current user.</i>
Auth: User		
DELETE	/v1/auth/sessions/{session_id}	<i>Revoke a specific session remotely, for example logging out an old phone.</i>
Auth: User		

Notes: this trio matters more for a fintech app than for most consumer apps -- when a user loses a phone, being able to see and individually kill active sessions from a new device is a basic, expected safety feature, not a nice-to-have.

3. User & KYC Endpoints

Maps to the USER and KYC_VERIFICATION tables from the schema document. The split between profile data the user edits themselves and identity verification that goes through a review process is reflected directly in having two separate sets of endpoints here.

3.1 Get and update own profile

GET	/v1/users/me	<i>Fetch the authenticated user's own profile.</i>
Auth: User		
Response 200 OK:		
<pre>{ "user_id": "8a6f3c2e-...", "username": "seyi_codes", "email": "seyi@example.com", "first_name": "Seyi", "last_name": "Adebayo", "city": "Lagos", "country_code": "NG", "default_currency_code": "NGN", "kyc_status": "verified", "two_factor_enabled": true }</pre>		

PATCH	/v1/users/me	<i>Update editable profile fields.</i>
Auth: User		

Notes: there is deliberately no `/v1/users/{user_id}` endpoint for arbitrary IDs exposed to ordinary client apps -- a regular user should only ever be able to fetch or modify their own profile, identified implicitly from their bearer token, never by passing someone else's ID in the URL. This is one of the most common authorization bugs in junior API implementations: building a get-by-id endpoint and checking only whether the caller is logged in, not whether the caller owns that specific id. The `/me` convention sidesteps the whole bug class by construction, since there is no id parameter to forget to check. PATCH uses partial-update semantics: only fields present in the request body are changed, everything else is left as-is, which is the defining behavior of PATCH versus PUT, where PUT would mean replacing the entire resource with exactly the given payload.

PATCH explicitly excludes fields that have their own dedicated, more carefully-guarded flow: email changes typically require re-verification through a confirmation link to the new address, and password changes go through Section 2.5's reset flow or a separate authenticated change-password endpoint, not a silent field update mixed in with changing a city.

3.2 KYC submission and status

POST	/v1/kyc/submissions	<i>Submit identity documents for verification.</i>
Auth: User		
Idempotent?: Recommended -- a slow upload retried by a flaky connection should not create two pending submissions.		

Request body:

```
{
  "document_type": "national_id",
  "document_reference": "tok_doc_9f3a2b...",
  "document_front_upload_id": "upl_881a...",
  "document_back_upload_id": "upl_881b...",
  "selfie_upload_id": "upl_881c..."
}
```

Response 202 Accepted:

```
{
  "kyc_id": "9c2e4f1a-...",
  "verification_status": "pending",
  "submitted_at": "2026-06-27T14:40:00Z"
}
```

Notes: the response status is 202 Accepted, not 200 OK or 201 Created -- this is a deliberate and useful signal. 202 means the server has received the request and queued it for processing, but the result is not ready yet, which is exactly true here, since identity verification typically runs through a third-party provider or manual review queue and finishes asynchronously, sometimes minutes later. The upload_id fields reference files uploaded separately via a dedicated file-upload endpoint, or a direct-to-storage presigned URL which is the more scalable pattern at any real volume, rather than embedding base64 image data directly in this JSON body -- base64-encoding images inflates payload size by roughly a third and makes request bodies unwieldy for no benefit.

GET	/v1/kyc/submissions	<i>List the current user's KYC submission history.</i>
Auth: User		
GET	/v1/kyc/submissions/{kyc_id}	<i>Fetch the status or result of a specific submission.</i>
Auth: User		

Notes: listing history, rather than only exposing current status, matters because a user may have a rejected attempt followed by an approved one, and being able to see why the first was rejected through a reason code or message your back-office review tool writes back helps them fix it on the second try instead of guessing.

3.3 KYC webhook from a third-party verification provider

POST	/v1/webhooks/kyc-provider	<i>Receive an asynchronous verification result from a third-party identity provider.</i>
Auth: Webhook (signed)		

Notes: covered in full in Section 10, but flagged here because it is the other half of 3.2's story -- the POST submission kicks the process off, and this webhook is how the result comes back once the third-party provider finishes its check, asynchronously, possibly minutes or hours later. The submission endpoint itself never blocks waiting for a verdict.

4. Account Endpoints

Maps to the ACCOUNT table. This is also a good section to introduce the standard list/detail/create resource pattern that repeats throughout the rest of the document -- once you've seen it here, Cards, Loans, and Invoices in later sections will feel like the same shape repeated with different fields.

4.1 List own accounts

GET	/v1/accounts	<i>List all accounts owned by the authenticated user.</i>
Auth: User		
Response 200 OK:		
<pre>{ "data": [{ "account_id": "acc_seyi_001", "account_type": "current", "account_number": "0123456789", "currency_code": "NGN", "balance_minor": 1240000, "account_status": "active" }, { "account_id": "acc_seyi_002", "account_type": "savings", "balance_minor": 5000000, "account_status": "active" }] }</pre>		

Notes: this list is implicitly scoped to the caller's user_id from the bearer token -- there is no user_id query parameter, for the same authorization reason given in Section 3.1. A user with three accounts (current, savings, investment wallet) gets all three back in one call; the client is responsible for picking which one to show by default or letting the user switch between them.

4.2 Get a single account

GET	/v1/accounts/{account_id}	<i>Fetch one account's current details and cached balance.</i>
Auth: User		

Notes: balance_minor returned here is the cached value described in the schema document's Section 7.3 -- fast to read, kept in sync by the ledger-posting routine on every transaction, and reconcilable against the ledger if it's ever in doubt. The endpoint does not recompute the balance from scratch on every call; that would make a screen that's opened dozens of times a day unnecessarily slow.

If account_id exists but doesn't belong to the caller, this returns 404 Not Found, not 403 Forbidden. This is a deliberate, common security convention: 403 confirms to an attacker that the resource exists but they're not allowed to see it, which is itself a small information leak (it

lets someone enumerate valid IDs even if they can't access the contents). 404 reveals nothing -- 'not found' is equally true whether the ID is wrong or just not yours.

4.3 Create a new account

POST	/v1/accounts	<i>Open a new account of a given type for the authenticated user.</i>
Auth: User + 2FA		
Idempotent?: Required		
Request body:		
<pre>{ "account_type": "savings", "currency_code": "NGN" }</pre>		
Response 201 Created:		
<pre>{ "account_id": "acc_seyi_003", "account_type": "savings", "account_number": "0123456802", "currency_code": "NGN", "balance_minor": 0, "account_status": "active", "created_at": "2026-06-27T15:00:00Z" }</pre>		

Notes: opening a financial account is the kind of action worth gating behind a fresh 2FA check, even though the user is already logged in -- it's a step up in consequence from merely viewing data. `account_number` is server-generated (following whatever local numbering scheme applies, such as Nigeria's NUBAN format), never client-supplied; allowing a client to propose its own account number would invite collisions and, worse, attempts to claim a number that looks like someone else's.

4.4 Update account metadata

PATCH	/v1/accounts/{account_id}	<i>Update mutable, non-financial account fields.</i>
Auth: User		
Request body (example -- renaming a savings goal target):		
<pre>{ "total_savings_minor": 10000000 }</pre>		

Notes: notice what is conspicuously not editable through this endpoint: `balance_minor`. There is no client-facing way to PATCH a balance directly, ever -- balance only changes as a side effect of posting transactions (Section 6), never as a direct field write. This is the API-level enforcement of the schema document's core rule that balance is ledger-derived, not a freely assignable column.

4.5 Freeze / close an account

POST	/v1/accounts/{account_id}/freeze	<i>Temporarily freeze an account (blocks new debits, allows credits).</i>
Auth: User + 2FA		
POST	/v1/accounts/{account_id}/close	<i>Permanently close an account.</i>
Auth: User + 2FA		

Notes: these are modeled as explicit action sub-resources (POST .../freeze, POST .../close) rather than as a PATCH that sets `account_status` directly. This is a deliberate API design choice worth understanding: a generic PATCH `{"account_status": "closed"}` looks simpler, but it invites a client to attempt any status transition, including invalid ones (reopening a closed account by PATCHing status back to active, for instance), pushing all the business-rule enforcement onto ad-hoc validation code. A dedicated action endpoint can enforce exactly which transitions are legal, return a specific, clear error for an illegal one (e.g. 409 Conflict, 'cannot close an account with a non-zero balance'), and is easier to find in API documentation than a side effect buried inside a generic field update. This pattern -- action sub-resources for anything that is a meaningful business event rather than a simple data edit -- shows up again for loans (disburse, repay) and invoices (pay, cancel) later in this document.

GET	/v1/accounts/{account_id}/balance-history	<i>Time-series of balance snapshots, for charting.</i>
-----	---	--

Auth: User

Notes: a dedicated endpoint for this (rather than asking the client to derive a chart from the full transaction list) exists because it can return pre-aggregated daily/weekly closing balances -- much smaller and faster for a client to render than replaying potentially thousands of individual transactions just to draw a line graph.

5. Card Endpoints

Maps to the CARD and CARD_PROVIDER tables. Every endpoint in this section inherits the PCI-DSS constraint established in the schema document: a raw card number is never accepted in a request body, never stored, and never returned in a response. The flow below is built around tokenization from end to end.

Why card creation looks different from every other POST in this document:

Most resources in this API accept the full data for the new resource directly in the POST body. Cards are the one place where that would be actively dangerous -- if your own server's API accepted a raw card number in a JSON field, that raw number would transit your network and briefly exist in your server's memory and logs, putting your whole infrastructure inside PCI's compliance scope. The standard fix, shown in Section 5.1, is for the card number to go directly from the user's device to the payment processor's own hosted field or SDK, which returns a token -- your API only ever receives and stores that token.

5.1 Add a new card

POST /v1/cards

Attach a tokenized card to an account.

Auth: User + 2FA

Idempotent?: Required

Request body:

```
{
  "account_id": "acc_seyi_001",
  "card_token": "tok_processor_4f9a2b1c...",
  "pan_bin": "539983",
  "pan_last_four": "4242",
  "cardholder_name": "SEYI ADEBAYO",
  "expiry_month": 9,
  "expiry_year": 2029,
  "provider_id": "prov_visa"
}
```

Response 201 Created:

```
{
  "card_id": "card_91a2...",
  "pan_bin": "539983",
  "pan_last_four": "4242",
  "card_status": "active",
  "created_at": "2026-06-27T15:10:00Z"
}
```

Notes: card_token, pan_bin, and pan_last_four arrive already produced by the payment processor's client-side SDK -- the device that captured the card number talked directly to the processor, not to your server, for that part. Your backend's job is only to record the token against the right account, never to see or transmit the full number itself. This is precisely the tokenization boundary described in the schema document, now expressed as an API contract: the request body's shape itself makes it structurally impossible to submit a full PAN, because there is no field for one.

5.2 List and view cards

GET	/v1/cards	List cards across all (or one) of the user's accounts.
Auth: User		
GET /v1/cards?account_id=acc_seyi_001		
Response 200 OK:		
<pre>{ "data": [{ "card_id": "card_91a2...", "provider_name": "Visa", "pan_last_four": "4242", "expiry_month": 9, "expiry_year": 2029, "card_status": "active" }] }</pre>		

GET	/v1/cards/{card_id}	Fetch one card's display-safe details.
Auth: User		

Notes: every response in this section, without exception, includes only the masked, PCI-approved display fields -- never anything that could be reassembled into a usable card number. There is no GET endpoint anywhere in this API that returns a full PAN, because the platform's own database never has one to return in the first place.

5.3 Block, unblock, and remove a card

POST	/v1/cards/{card_id}/block	Immediately block a card from further use, for a lost or stolen card.
Auth: User		
POST	/v1/cards/{card_id}/unblock	Reverse a block.
Auth: User + 2FA		
DELETE	/v1/cards/{card_id}	Permanently remove a card from the account.
Auth: User + 2FA		

Notes: blocking is intentionally easier to trigger than unblocking -- plain User auth for block, but User + 2FA for unblock or removal. If a card is genuinely lost or stolen, the user should be able to kill it in one tap, under stress, without fumbling for a second factor. Reversing that decision is exactly the kind of action worth slowing down if an attacker, rather than the genuine owner, gained access to the account, since unblocking a card is a necessary step before it could be misused again.

5.4 Card transaction controls

PATCH	/v1/cards/{card_id}/controls	Update spending controls: limits, merchant category restrictions, online and ATM toggles.
Auth: User + 2FA		
Request body:		
{		

```
"daily_limit_minor": 5000000,  
"online_payments_enabled": true,  
"atm_withdrawals_enabled": false  
}
```

Notes: this is its own endpoint, separate from a general card update, because spending controls are conceptually a different kind of data from card identity -- they change frequently, are read by the authorization-decision path on every single transaction attempt, and arguably deserve their own audit trail given they directly gate how much money can move. Grouping fast-changing, security-sensitive settings into their own resource is a pattern worth recognizing -- the same instinct that gave the loan repayment schedule its own table instead of cramming installment data onto the loan row in the schema document.

6. Transaction & Ledger Endpoints

Maps to TRANSACTION and LEDGER_ENTRY -- the architectural core of both the schema and, correspondingly, the API. Every other money-moving section in this document, loans and invoices included, ultimately funnels through the endpoints here, so it is worth reading this section slowly even if you skim the rest.

6.1 Create a transfer

POST	/v1/transactions	<i>Create a transfer between two accounts.</i>
Auth: User		
Idempotent?: Required -- this is the canonical example the whole Idempotency-Key convention in Section 1.3 exists for.		
POST /v1/transactions Idempotency-Key: 8a6f3c2e-9b1d-4f5a-8c3e-7d2a1b9f0c4e Request body: <pre>{ "transaction_type": "transfer", "sender_account_id": "acc_seyi_001", "receiver_account_id": "acc_feranmi_009", "amount_minor": 500000, "currency_code": "NGN", "description": "August rent contribution" }</pre>		
Response 201 Created: <pre>{ "transaction_id": "txn_8841...", "status": "settled", "amount_minor": 500000, "currency_code": "NGN", "created_at": "2026-06-27T15:20:00Z", "settled_at": "2026-06-27T15:20:01Z" }</pre>		

Notes: this single endpoint, internally, triggers exactly the sequence documented step by step in the schema document: check the idempotency key, open a database transaction, write the TRANSACTION row, write a matching debit and credit pair to LEDGER_ENTRY, update both accounts' cached balances, mark the transaction settled, commit. None of that internal sequencing is visible to the API caller -- from the outside, it is one POST and one response -- which is exactly the point of encapsulating it behind a single endpoint rather than exposing ledger entries as something a client posts to directly.

Why there is no public endpoint for creating ledger entries directly:

LEDGER_ENTRY rows are never created directly by a client request. They are always a side effect of a higher-level business action -- a transfer, a card payment settling, a loan disbursement. Exposing ledger-entry creation as its own public endpoint would let a buggy or malicious client write an unbalanced entry, a debit with no matching credit, which is exactly the integrity guarantee the whole double-entry pattern exists to prevent. The general rule: if writing to a table directly could violate an invariant the system depends on, do not expose a direct write endpoint for that table at all -- only expose the higher-level action that writes to it correctly, every

time, as an internal implementation detail.

6.2 List and search transactions

GET	/v1/accounts/{account_id}/transactions	<i>List transaction history for one account, paginated.</i>
Auth: User		
<pre>GET /v1/accounts/acc_seyi_001/transactions?limit=20&after=eyJpZCI6IjQ0N30 &from_date=2026-06-01&to_date=2026-06-30&status=settled</pre>		
Response 200 OK: <pre>{ "data": [{ "transaction_id": "txn_8841...", "transaction_type": "transfer", "direction": "debit", "counterparty_account_id": "acc_feranmi_009", "amount_minor": 500000, "status": "settled", "created_at": "2026-06-27T15:20:00Z" }], "next_cursor": "eyJpZCI6IjQ0OCJ9", "has_more": true }</pre>		

Notes: direction (debit or credit, from this account's point of view) and counterparty_account_id are derived, presentational fields, computed by the API layer from the underlying sender and receiver pair based on which side the account_id in the URL is on -- the client should not have to do that comparison itself on every row just to render a statement. This is a good example of an API response intentionally not mirroring the database schema one to one: the schema stores sender and receiver because that is symmetric and storage-efficient, but the API exposes direction and counterparty because that is what is actually useful to a client rendering a list.

GET	/v1/transactions/{transaction_id}	<i>Fetch full detail for a single transaction.</i>
Auth: User (must be sender or receiver)		
Response 200 OK: <pre>{ "transaction_id": "txn_8841...", "transaction_type": "transfer", "sender_account_id": "acc_seyi_001", "receiver_account_id": "acc_feranmi_009", "amount_minor": 500000, "currency_code": "NGN", "status": "settled", "description": "August rent contribution", "ledger_entries": [{ "entry_id": "le_19031", "account_id": "acc_seyi_001", "direction": "DEBIT", "amount_minor": 500000 }, { "entry_id": "le_19032", "account_id": "acc_feranmi_009", "direction": "CREDIT", "amount_minor": 500000 }] }</pre>		

Notes: the single-transaction detail view is one of the few places it is worth exposing the underlying ledger entries directly -- it is genuinely useful for a transaction-details screen to show the double-entry breakdown, useful for support staff debugging a dispute and satisfying for a user who wants to understand exactly what happened, whereas exposing the same level of raw detail in the list endpoint above would be noisy and unnecessary.

6.3 Reverse or refund a transaction

POST	/v1/transactions/{transaction_id}/reverse	<i>Reverse a settled transaction by posting an equal and opposite entry.</i>
Auth: User + 2FA, or Admin depending on the reversal reason		
Idempotent?: Required		
Response 201 Created:		
<pre>{ "reversal_transaction_id": "txn_8855...", "original_transaction_id": "txn_8841...", "status": "settled" }</pre>		

Notes: this returns 201 Created, not 200 OK, because a reversal is implemented as a brand new transaction, with its own new ledger entries undoing the original, not as an edit to the old one -- directly enforcing the schema document's immutability rule that ledger rows are never updated or deleted, only ever offset by a new, opposite entry. The original transaction's status moves to reversed, but the original rows themselves stay exactly as they were, fully visible in history. This is also why the endpoint is a sub-resource action rather than a DELETE on the transaction -- DELETE would imply the record disappears, which is precisely what must never happen to a financial record.

6.4 Idempotency key lookup, a debugging aid

GET	/v1/idempotency-keys/{key}	<i>Look up what happened for a given idempotency key.</i>
Auth: User (scoped to keys the caller created), or Admin		

Notes: a small but genuinely useful endpoint for both client-side debugging and your own support tooling -- knowing exactly what the server did the last time a given key was sent is worth being able to answer directly, rather than only inferring it from whether a transaction eventually shows up. Several real payment platforms, Stripe included, expose exactly this kind of lookup.

7. Loan Endpoints

Maps to LOAN and LOAN_REPAYMENT_SCHEDULE. The interesting design decision in this section is the lifecycle: a loan moves through distinct states, application, approval, disbursement, repayment, closure, and the endpoints are deliberately shaped around that lifecycle rather than around generic CRUD.

7.1 Apply for a loan

POST	/v1/loans/applications	<i>Submit a loan application for review.</i>
Auth: User		
Idempotent?: Recommended		
Request body:		
<pre>{ "account_id": "acc_seyi_001", "loan_type": "personal", "requested_amount_minor": 20000000, "currency_code": "NGN", "purpose": "Laptop for freelance work" }</pre>		
Response 202 Accepted:		
<pre>{ "application_id": "loanapp_771a...", "status": "pending_review", "submitted_at": "2026-06-27T15:30:00Z" }</pre>		

Notes: this creates an application, not a LOAN row directly -- a request for credit and an approved, money-bearing loan are different things and deserve different resources. Modeling them as the same resource with a pending status field is tempting because it feels like less work, but it blurs a distinction that matters: an application has no principal, no repayment schedule, and no ledger relationship at all until a decision is made, whereas an active loan has all three. Once approved, a real loan resource is created, shown in Section 7.2, that references back to the application that led to it.

7.2 Loan decision and disbursement

GET	/v1/loans/applications/{application_id}	<i>Check the status of a loan application.</i>
Auth: User		
POST	/v1/loans/applications/{application_id}/approve	<i>Approve an application and create the resulting loan, an internal credit and risk decision.</i>
Auth: Admin or System, such as an automated credit-scoring service		
POST	/v1/loans/{loan_id}/disburse	<i>Release approved loan funds into the borrower's account.</i>
Auth: Admin or System		
Idempotent?: Required		

Response 201 Created (disbursement):

```
{
  "transaction_id": "txn_9001...",
  "loan_id": "loan_551b...",
  "amount_minor": 20000000,
  "status": "settled"
}
```

Notes: disbursement is deliberately its own action endpoint, and under the hood it calls exactly the same internal transaction-creation logic from Section 6.1 -- it produces a transaction row of type loan_disbursement and a matching ledger entry pair, just triggered by an internal approval workflow instead of a user-initiated transfer. This is a good illustration of a broader principle: loans, invoices, and card settlements are not separate money-movement mechanisms competing with the ledger; they are all just different entry points into the same underlying transaction-and-ledger machinery from Section 6.

7.3 View loan details and repayment schedule

GET	/v1/loans	<i>List the authenticated user's loans.</i>
Auth: User		
GET	/v1/loans/{loan_id}	<i>Fetch one loan's details, including the current remaining balance.</i>
Auth: User		
GET	/v1/loans/{loan_id}/schedule	<i>Fetch the full amortization schedule.</i>
Auth: User		

Response 200 OK (schedule):

```
{
  "data": [
    {
      "installment_number": 1,
      "due_date": "2026-07-27",
      "installment_amount_minor": 1750000,
      "paid_flag": false
    },
    { "installment_number": 2, "due_date": "2026-08-27", "installment_amount_minor": 1750000, "paid_flag": false }
  ]
}
```

Notes: this is a direct API expression of why the loan repayment schedule exists as its own table in the schema -- without it, there would be no way to answer what is due and when at all, only a single running balance. A schedule endpoint is what turns a loan feature from you-owe-some-amount into something a user can actually plan around.

7.4 Make a repayment

POST	/v1/loans/{loan_id}/repayments	<i>Make a repayment against a loan.</i>
Auth: User		
Idempotent?: Required		
Request body:		
{		
"source_account_id": "acc_seyi_001",		

```
"amount_minor": 1750000
}

Response 201 Created:
{
  "transaction_id": "txn_9050...",
  "loan_id": "loan_551b...",
  "schedule_installment_marked_paid": 1,
  "new_balance_remaining_minor": 18250000
}
```

Notes: a single repayment call does three things atomically: posts a transaction and ledger pair, debiting the user's account and crediting the loan-holding side, marks the corresponding schedule row's paid flag true, and recalculates the remaining balance. All three happen inside one database transaction, for the same reason the transfer in Section 6.1 does -- a partial failure, where the payment posts but the schedule row never gets marked paid, would leave the system in a state that looks, to a reconciliation job, exactly like a bug, because it is one.

8. Invoice Endpoints

Maps to INVOICE. The smallest section in this document, deliberately -- once you understand the action-sub-resource pattern from Sections 6 and 7, invoices are a straightforward application of the same idea: create, list, view, and a small set of explicit state-transition actions.

8.1 Create and list invoices

POST	/v1/invoices	Create an invoice billed to an account.
Auth: User, or System for a recurring billing job		
Idempotent?: Recommended		
Request body:		
<pre>{ "account_id": "acc_seyi_001", "amount_due_minor": 850000, "currency_code": "NGN", "due_date": "2026-07-15" }</pre>		
Response 201 Created:		
<pre>{ "invoice_id": "inv_2231...", "invoice_status": "open", "amount_due_minor": 850000, "due_date": "2026-07-15" }</pre>		
GET	/v1/invoices	List invoices for the authenticated user's accounts.
Auth: User		
GET /v1/invoices?status=open&limit=20		
GET	/v1/invoices/{invoice_id}	Fetch a single invoice's details.
Auth: User		

8.2 Pay and cancel an invoice

POST	/v1/invoices/{invoice_id}/pay	Settle an invoice using funds from a specified account.
Auth: User		
Idempotent?: Required		
Request body:		
<pre>{ "source_account_id": "acc_seyi_001" }</pre>		
Response 200 OK:		
<pre>{ "invoice_id": "inv_2231...", "invoice_status": "paid", "settling_transaction_id": "txn_9102..." }</pre>		
POST	/v1/invoices/{invoice_id}/cancel	Cancel an unpaid invoice.
Auth: User or Admin		

Notes: `pay` is, once again, not a generic field update -- it is an action that internally creates a real transaction, exactly mirroring how loan repayments work in Section 7.4, and links it back via a `settling_transaction_id` field. `cancel` is only legal while the invoice is still open; attempting to cancel an already-paid invoice should return 409 Conflict with a clear error message, not silently succeed or silently fail. This is worth calling out explicitly because it is a common interview question in disguise -- asking what status code you would return for cancelling an order that has already shipped is really asking whether you understand that not every action is legal in every state, and that the API needs to say so clearly rather than leaving the client to guess.

9. Investment & Brokerage Endpoints

Maps to SECURITY, INVESTMENT_HOLDING, and EXTERNAL_BROKERAGE_LINK. This section is split cleanly into two halves: endpoints for connecting and syncing an external brokerage account, and endpoints for reading the resulting holdings, matching the original goal of pulling investment data live from a third-party platform via API.

9.1 Connect an external brokerage

POST	/v1/brokerage-links	<i>Begin connecting an external brokerage or investment account.</i>
Auth: User + 2FA		
Request body:		
<pre>{ "provider_name": "example_brokerage" }</pre>		
Response 200 OK:		
<pre>{ "authorization_url": "https://example-brokerage.com/oauth/authorize?client_id=...", "state": "9f3a2b1c..." }</pre>		

Notes: this does not create the link directly -- it returns an OAuth authorization URL the client redirects the user to, since the user needs to log into the brokerage's own site and explicitly grant access. The actual link only gets created once the brokerage redirects back with an authorization code, handled by the callback endpoint below. This two-step shape is not a stylistic choice, it is a structural requirement of how OAuth works: your server can never see the user's brokerage password, only a code the brokerage hands back after the user authenticates directly with them.

GET	/v1/brokerage-links/oauth/callback	<i>OAuth redirect target; exchanges the authorization code for a token and creates the link.</i>
Auth: Public, protected by the unguessable state value and the OAuth code itself		
GET	/v1/brokerage-links	<i>List the user's connected brokerage links.</i>
Auth: User		
Response 200 OK:		
<pre>{ "data": [{ "link_id": "link_44a1...", "provider_name": "example_brokerage", "link_status": "active", "last_synced_at": "2026-06-27T13:00:00Z" }] }</pre>		

Notes: this list never includes the OAuth token reference or anything resembling the underlying token -- the API surface mirrors the schema document's instruction that even the reference value is a handle into a secrets vault, not something to ever expose back to a client, even the

legitimate owner's own client. There is simply no GET response anywhere in this API that returns token material.

DELETE `/v1/brokerage-links/{link_id}`

Disconnect a brokerage link and revoke its token.

Auth: User + 2FA

9.2 Sync and view holdings

POST `/v1/brokerage-links/{link_id}/sync`

Manually trigger an on-demand sync, in addition to the scheduled background job.

Auth: User

Idempotent?: Not strictly required -- a sync is naturally safe to repeat, since it always upserts the latest state rather than appending new rows.

Response 202 Accepted:

```
{
  "link_id": "link_44a1...",
  "sync_status": "queued"
}
```

Notes: this returns 202 Accepted and immediately enqueues a job onto the same distributed job queue infrastructure described elsewhere, rather than calling the brokerage's API synchronously inside the HTTP request. A third-party brokerage API is exactly the kind of dependency that can be slow or rate-limited at unpredictable moments; if this endpoint blocked on it directly, one slow upstream call would tie up a request thread and degrade the experience for everyone else hitting the API at the same time. This is the same reasoning the schema document gave for why the scheduled sync job belongs in a queue in the first place -- this endpoint is just the manual, on-demand trigger for the same underlying job.

GET `/v1/holdings`

List the authenticated user's investment holdings across all accounts.

Auth: User

Response 200 OK:

```
{
  "data": [
    {
      "holding_id": "hold_661c...",
      "security": { "ticker_symbol": "DANGCEM", "company_name": "Dangote Cement Plc",
"asset_type": "stock" },
      "quantity": "150.000000",
      "average_cost_minor": 28500,
      "last_synced_at": "2026-06-27T13:00:00Z"
    }
  ]
}
```

Notes: the response nests a security object inline rather than only returning a bare security_id -- a deliberate denormalization at the API layer, distinct from the database layer, where SECURITY stays properly normalized. A client rendering a portfolio screen needs the ticker and company name on every single row; forcing it to make a second round-trip per holding just to resolve that, or to batch-fetch every distinct security itself, would be needless friction for a value that almost never changes. This is the API-design equivalent of the schema's own guidance to

denormalize deliberately, after identifying a real client-side need -- here, the need is obvious and immediate, so it is reasonable to bake the join into the response from day one rather than waiting to discover the need.

GET	/v1/holdings/{holding_id}	Fetch a single holding's full detail and price history, if available.
-----	---------------------------	---

Auth: User

10. Webhook Endpoints (Incoming, From External Providers)

Every endpoint so far in this document has been your API responding to a request your own users or your own internal services initiated. Webhooks are the mirror image: an external system, a payment processor, a brokerage, an identity verification provider, calling into your API on its own schedule to tell you something happened. This section covers what that looks like and how to secure it -- a small set of endpoints, but a conceptually distinct one worth its own section.

10.1 The shared webhook security pattern

Every webhook endpoint in this section follows the same authentication approach, distinct from the bearer-token pattern used everywhere else in this document: signature verification. The external provider signs its request body using a shared secret both sides know, and includes the signature in a header; your endpoint recomputes the signature from the raw request body using that same secret and rejects the request if the two do not match.

```
POST /v1/webhooks/payment-processor
X-Signature: t=1719500000,v1=5257a869e7bcd5e...
Content-Type: application/json

{ "event": "charge.settled", "data": { "card_token": "tok...", "amount_minor": 250000 } }
```

Notes: this is why webhook endpoints get their own Auth label of Webhook (signed) in Section 1.2's table rather than being marked Public -- they are unauthenticated in the sense that there is no user session involved, but they are absolutely not open to arbitrary callers. A request without a valid signature is rejected with 401 Unauthorized before any business logic runs, exactly like a missing bearer token would be rejected elsewhere. Most providers also include a timestamp in the signed payload specifically so a captured, valid signature from months ago cannot be replayed indefinitely -- verification logic should reject signatures older than a few minutes even if the signature itself is technically valid.

Webhooks must be idempotent too, for a different reason than client retries:

External providers retry webhook delivery on their own, independent of anything your client does -- if your server returns anything other than a prompt success response, most providers will re-send the same event later, sometimes repeatedly. This means every webhook handler needs its own deduplication, typically keyed on the provider's own event ID rather than a client-supplied idempotency key, since there is no client here in the usual sense. Store and check the event ID before processing, exactly like the idempotency-key pattern from Section 1.3, just with the key sourced from the provider's payload instead of a request header.

10.2 Payment processor webhook

POST	/v1/webhooks/payment-processor	<i>Receive asynchronous card charge, settlement, and dispute events.</i>
Auth: Webhook (signed)		

Example payloads (event field varies):

```
{ "event": "charge.settled", "data": { "processor_charge_id": "...", "card_token":
"...", "amount_minor": 250000 } }
{ "event": "charge.failed", "data": { "processor_charge_id": "...", "failure_reason":
"insufficient_funds" } }
{ "event": "dispute.opened", "data": { "processor_charge_id": "...", "reason":
"fraudulent" } }
```

Notes: charge.settled is what actually completes a card payment transaction that was left in pending status when the user first initiated it. Section 6.1's flow assumed an instant transfer; a card charge realistically often does not settle instantly, since it depends on an external authorization network. This webhook is what moves that transaction's status from pending to settled and only then writes the ledger entries -- worth noting as a variation on the schema document's ledger-posting routine: for card payments specifically, ledger entries are written on settlement confirmation, not at the moment the charge is initiated.

10.3 KYC and identity verification provider webhook

POST /v1/webhooks/kyc-provider

Receive a verification result for a previously submitted KYC document.

Auth: Webhook (signed)

```
{ "event": "verification.completed", "data": { "kyc_id": "9c2e4f1a-...", "result":
"approved" } }
```

Notes: covered briefly already in Section 3.3; included again here for completeness, since this section is meant to be a complete catalog of every webhook the platform receives, not just a conceptual explanation.

10.4 Brokerage webhook, where supported

POST /v1/webhooks/brokerage

Receive push notifications of holding changes, where a brokerage supports webhooks instead of or alongside polling.

Auth: Webhook (signed)

Notes: not every brokerage offers webhooks -- many only offer a pollable REST API, which is why Section 9.2's scheduled sync job exists as the primary mechanism. Where a webhook is available, it is strictly an optimization layered on top of polling, letting a holding update propagate in near-real-time instead of waiting for the next scheduled sync, not a replacement for it -- the polling job remains the system's source of truth and safety net in case a webhook delivery is ever missed.

11. Admin & Compliance Endpoints

Maps to AUDIT_LOG plus internal review queues for KYC and loans that were referenced earlier but not detailed. These endpoints are never exposed to the consumer-facing mobile or web client at all -- they sit behind a completely separate authentication and authorization system, internal staff accounts with their own roles rather than USER rows, and are often not even reachable from the public internet, only from an internal network or VPN.

11.1 Audit log access

GET	/v1/admin/audit-logs	<i>Search audit log entries by entity, actor, or time range.</i>
Auth: Admin		
<pre>GET /v1/admin/audit-logs?entity_type=account&entity_id=acc_seyi_001 &from_date=2026-06-01&limit=50</pre>		
Response 200 OK:		
<pre>{ "data": [{ "audit_id": "aud_5571...", "actor_id": "staff_user_22", "entity_type": "account", "entity_id": "acc_seyi_001", "action": "status_change", "before_state": { "account_status": "active" }, "after_state": { "account_status": "frozen" }, "created_at": "2026-06-20T09:15:00Z" }] }</pre>		

Notes: this is a read-only endpoint, deliberately -- there is no update or delete anywhere in this section for the audit log, because allowing edits to the audit trail defeats its entire purpose. An audit log you can edit is not an audit log. The same immutability principle that governs LEDGER_ENTRY in the schema document applies here for an identical reason.

11.2 KYC review queue

GET	/v1/admin/kyc/queue	<i>List pending KYC submissions awaiting manual review.</i>
Auth: Admin		
POST	/v1/admin/kyc/submissions/{kyc_id}/decide	<i>Record a manual approve or reject decision.</i>
Auth: Admin		
Request body: <pre>{ "decision": "rejected", "reason": "Document image was blurry, ask the user to resubmit." }</pre>		

Notes: this is the human-in-the-loop counterpart to the automated webhook in Section 10.3 -- some submissions get auto-approved or auto-rejected by a third-party verification provider, but edge cases, a borderline match or a flagged document, fall through to a back-office reviewer

using this endpoint. Both paths converge on the same underlying KYC verification row update, which is exactly why the schema document includes a `verified_by` field -- it records whether a given decision came from a person or a system.

11.3 Loan underwriting queue

GET	/v1/admin/loans/applications	<i>List pending loan applications for review.</i>
Auth: Admin		

Notes: this is the back-office view of the same application resource introduced in Section 7.1, now exposed with whatever additional risk and credit data an underwriter needs to see that an ordinary user-facing GET would never include, such as a credit score, a debt-to-income calculation, or flags from a fraud model. It is worth noticing this is the same underlying entity exposed through two completely different endpoints, shaped for two completely different audiences -- a recurring, healthy pattern, rather than trying to make one endpoint serve both a customer and an underwriter by stuffing every possible field into a single response and gating pieces of it with field-level permissions.

11.4 Reconciliation

POST	/v1/admin/reconciliation/run	<i>Manually trigger a full reconciliation pass.</i>
Auth: Admin, or System for the scheduled job that calls this internally on a timer		

GET	/v1/admin/reconciliation/reports/{report_id}	<i>Fetch the results of a reconciliation run.</i>
Auth: Admin		

Response 200 OK:

```
{
  "report_id": "recon_2026-06-27",
  "accounts_checked": 14302,
  "discrepancies_found": 0,
  "status": "clean"
}
```

Notes: this is the API surface for the reconciliation job described in the schema document's discussion of standing reconciliation processes -- the job itself runs on a schedule via the same job queue infrastructure used for brokerage sync, but having an endpoint to trigger it on demand, after a suspicious support ticket for instance, and to fetch a specific run's results is what makes the process actually usable by a human, rather than something that only ever runs invisibly in the background.

12. Full Endpoint Index

A single scannable table of every endpoint in this document, in the order they appear above. Use this as a quick reference once you have read the detailed sections at least once -- it is also a reasonable starting checklist for an OpenAPI/Swagger spec, which is the natural next artifact to produce once this design is settled, covered in Section 15.

12.1 Authentication

Section 2

Method	Path	Auth	Summary
POST	/v1/auth/register	Public	Create a new user.
POST	/v1/auth/login	Public	Exchange credentials for tokens.
POST	/v1/auth/refresh	Public	Exchange refresh token for new access token.
POST	/v1/auth/2fa/enable	User	Begin 2FA setup.
POST	/v1/auth/2fa/verify	User	Confirm a TOTP code.
DELETE	/v1/auth/2fa	User+2FA	Disable 2FA.
POST	/v1/auth/password/forgot	Public	Request password reset.
POST	/v1/auth/password/reset	Public	Complete password reset.
POST	/v1/auth/logout	User	Revoke current refresh token.
GET	/v1/auth/sessions	User	List active sessions.
DELETE	/v1/auth/sessions/{session_id}	User	Revoke a specific session.

12.2 User & KYC

Section 3

Method	Path	Auth	Summary
GET	/v1/users/me	User	Fetch own profile.
PATCH	/v1/users/me	User	Update own profile.
POST	/v1/kyc/submissions	User	Submit identity documents.
GET	/v1/kyc/submissions	User	List own KYC history.
GET	/v1/kyc/submissions/{kyc_id}	User	Fetch one submission's status.

12.3 Accounts

Section 4

Method	Path	Auth	Summary
GET	/v1/accounts	User	List own accounts.

Method	Path	Auth	Summary
GET	/v1/accounts/{account_id}	User	Fetch one account.
POST	/v1/accounts	User+2FA	Open a new account.
PATCH	/v1/accounts/{account_id}	User	Update account metadata.
POST	/v1/accounts/{account_id}/freeze	User+2FA	Freeze an account.
POST	/v1/accounts/{account_id}/close	User+2FA	Close an account.
GET	/v1/accounts/{account_id}/balance-history	User	Time-series balance snapshots.

12.4 Cards

Section 5

Method	Path	Auth	Summary
POST	/v1/cards	User+2FA	Attach a tokenized card.
GET	/v1/cards	User	List cards.
GET	/v1/cards/{card_id}	User	Fetch one card.
POST	/v1/cards/{card_id}/block	User	Block a card.
POST	/v1/cards/{card_id}/unblock	User+2FA	Unblock a card.
DELETE	/v1/cards/{card_id}	User+2FA	Remove a card.
PATCH	/v1/cards/{card_id}/controls	User+2FA	Update spending controls.

12.5 Transactions & Ledger

Section 6

Method	Path	Auth	Summary
POST	/v1/transactions	User	Create a transfer.
GET	/v1/accounts/{account_id}/transactions	User	List account transaction history.
GET	/v1/transactions/{transaction_id}	User	Fetch one transaction's full detail.
POST	/v1/transactions/{transaction_id}/reverse	User+2FA/ Admin	Reverse a settled transaction.
GET	/v1/idempotency-keys/{key}	User/Admin	Look up an idempotency key's result.

12.6 Loans

Section 7

Method	Path	Auth	Summary
POST	/v1/loans/applications	User	Apply for a loan.

Method	Path	Auth	Summary
GET	/v1/loans/applications/{application_id}	User	Check application status.
POST	/v1/loans/applications/{application_id}/approve	Admin/System	Approve and create a loan.
POST	/v1/loans/{loan_id}/disburse	Admin/System	Disburse approved loan funds.
GET	/v1/loans	User	List own loans.
GET	/v1/loans/{loan_id}	User	Fetch one loan's detail.
GET	/v1/loans/{loan_id}/schedule	User	Fetch amortization schedule.
POST	/v1/loans/{loan_id}/repayments	User	Make a repayment.

12.7 Invoices

Section 8

Method	Path	Auth	Summary
POST	/v1/invoices	User/System	Create an invoice.
GET	/v1/invoices	User	List invoices.
GET	/v1/invoices/{invoice_id}	User	Fetch one invoice.
POST	/v1/invoices/{invoice_id}/pay	User	Pay an invoice.
POST	/v1/invoices/{invoice_id}/cancel	User/Admin	Cancel an unpaid invoice.

12.8 Investments & Brokerage

Section 9

Method	Path	Auth	Summary
POST	/v1/brokerage-links	User+2FA	Begin connecting a brokerage.
GET	/v1/brokerage-links/oauth/callback	Public, state-protected	OAuth redirect target.
GET	/v1/brokerage-links	User	List connected brokerages.
DELETE	/v1/brokerage-links/{link_id}	User+2FA	Disconnect a brokerage.
POST	/v1/brokerage-links/{link_id}/sync	User	Trigger a manual sync.
GET	/v1/holdings	User	List investment holdings.
GET	/v1/holdings/{holding_id}	User	Fetch one holding's detail.

12.9 Webhooks

Section 10

Method	Path	Auth	Summary
POST	/v1/webhooks/payment-processor	Webhook (signed)	Card charge, settlement, dispute events.
POST	/v1/webhooks/kyc-provider	Webhook (signed)	Identity verification results.
POST	/v1/webhooks/brokerage	Webhook (signed)	Holding change push notifications.

12.10 Admin & Compliance

Section 11

Method	Path	Auth	Summary
GET	/v1/admin/audit-logs	Admin	Search audit log entries.
GET	/v1/admin/kyc/queue	Admin	List pending KYC submissions.
POST	/v1/admin/kyc/submissions/{kyc_id}/decide	Admin	Record a manual KYC decision.
GET	/v1/admin/loans/applications	Admin	List pending loan applications.
POST	/v1/admin/reconciliation/run	Admin/ System	Trigger a reconciliation pass.
GET	/v1/admin/reconciliation/reports/{report_id}	Admin	Fetch a reconciliation report.

13. Error Format Reference

Section 1.6 introduced the RFC 9457 problem+json shape every error in this API uses. This section is the complete reference: every standard HTTP status code this API actually uses and what it means here specifically, plus the catalog of error type values that recur across multiple endpoints, rather than repeating the same error description in every section above.

13.1 HTTP status codes used in this API

Status code reference

Code	Name	When this API uses it
200	OK	A GET, PATCH, or action endpoint succeeded and is returning a complete representation of the result immediately.
201	Created	A POST created a new resource, a transaction, a card, an account, and the response body represents that new resource.
202	Accepted	The request was valid and has been queued, but the result is not ready yet -- KYC submission, brokerage sync trigger, reconciliation run.
204	No Content	The request succeeded and there is nothing meaningful to return. Used sparingly in this API, since most successful actions here return at least a small confirmation body, as a fintech client usually wants to see the resulting state immediately.
400	Bad Request	The request is malformed independent of any business rule -- invalid JSON, a missing required field, a missing Idempotency-Key on an endpoint that requires one.
401	Unauthorized	No valid credential was presented at all -- missing or expired bearer token, invalid webhook signature.
403	Forbidden	A valid credential was presented, but it is not allowed to perform this specific action. Used sparingly -- see the discussion in Section 4.2 of why this API often prefers 404 for resource-ownership mismatches.
404	Not Found	The resource does not exist, or exists but does not belong to the caller, per Section 4.2.
409	Conflict	The request is well-formed but conflicts with the resource's current state -- cancelling an already-paid invoice, unblocking a card that was never blocked, a concurrent request reusing an idempotency key that is still mid-flight.
422	Unprocessable Content	The request is syntactically valid JSON but fails a business validation rule -- insufficient funds, an idempotency key reused with a different payload, an amount of zero or less.
429	Too Many Requests	The caller has exceeded its rate limit, per Section 1.7. Always paired with a Retry-After header.
500	Internal Server Error	An unexpected failure on the server's side. The response body still follows the problem+json shape, but the detail field should be generic and never leak internal stack traces or query text to the client.
503	Service Unavailable	A required downstream dependency, a payment processor or a

Code	Name	When this API uses it
		brokerage API, is down. Distinguishing this from a generic 500 helps a client decide whether retrying later is likely to help.

13.2 Standard error type catalog

Each error's type field is a stable, URL-shaped identifier a client can match on programmatically, far more reliable than parsing the human-readable title or detail string, which may change wording over time without notice. The table below lists the values that recur across multiple sections of this document.

Common error types

type (suffix after /errors/)	Typical status	Meaning
insufficient-funds	422	An account's balance is lower than the amount a transaction would debit.
idempotency-key-required	400	A mutating financial endpoint was called without an Idempotency-Key header.
idempotency-key-conflict	422	The same Idempotency-Key was reused with a different request payload.
idempotency-key-in-flight	409	A request with this Idempotency-Key is still being processed; retry shortly.
invalid-state-transition	409	The requested action is not legal given the resource's current status, for example cancelling a paid invoice.
kyc-required	403	The action requires a verified KYC status and the caller's account has not completed verification.
two-factor-required	401	The endpoint requires a fresh 2FA confirmation in addition to the bearer token.
card-blocked	422	An operation was attempted using a card whose status is not active.
rate-limit-exceeded	429	The caller has exceeded its allotted request rate.
validation-error	400	One or more request fields failed basic shape, type, or range validation; the details field lists each one.
webhook-signature-invalid	401	An incoming webhook's signature did not verify against the expected secret.

13.3 Validation error detail shape

When multiple fields fail validation simultaneously, a common case on registration or account-creation forms, the response includes a details array so a client can highlight every problem at once rather than forcing a slow one-error-at-a-time correction loop:

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json

{
  "type": "https://api.yourplatform.com/errors/validation-error",
  "title": "Validation failed",
  "status": 400,
  "detail": "One or more fields failed validation.",
  "request_id": "req_3f9a2b1c",
  "details": [
    { "field": "email", "issue": "must be a valid email address" },
    { "field": "date_of_birth", "issue": "must indicate an age of 18 or older" }
  ]
}
```

Notes: `request_id` appears on every error response in this API, success or failure, and should also appear in your server-side structured logs for that request. This is what turns a vague user report of something going wrong into a debuggable problem in under a minute -- ask for the `request_id`, search the logs, see exactly what happened, instead of trying to reconstruct a guess from a vague description.

14. Versioning, Rate Limiting & Deprecation Strategy

Section 1.1 and 1.7 introduced versioning and rate limiting briefly as conventions; this section goes deeper into the reasoning, since these are exactly the kind of topic an interviewer will push on with follow-up questions, and saying you picked v1 because it is common is a weaker answer than understanding what versioning is actually protecting against.

14.1 Versioning is about isolation, not the version number itself

The detail that matters more than whether you write a v1 path segment or a date-based header: once a version is published, every field a client could plausibly depend on becomes something you have implicitly promised to keep working. This is sometimes called Hyrum's Law -- with enough consumers of an API, every observable behavior of the system will eventually be depended on by somebody, whether or not that behavior was ever documented as a guarantee. A field you added almost as an afterthought can become load-bearing for a client you do not know about, simply because it was there and they used it.

Practical consequences for how this API evolves over time:

- Adding a new, optional field to a response is always safe and never requires a version bump -- existing clients that do not know about the field simply ignore it.
- Adding a new required field to a request is a breaking change -- existing clients do not send it, and a naive implementation would start rejecting requests that used to work. If a field must become required, it ships in v2, with v1 either defaulting it server-side or continuing to accept its absence indefinitely.
- Changing a field's type, a string becoming an object or an integer becoming a string, is always a breaking change, no exceptions, even if it feels like a minor cleanup on your end.
- Removing a field, even one that seems unused, is breaking until proven otherwise -- the safe default is to deprecate it, covered in Section 14.3, and watch usage metrics before ever actually removing it.

The practical takeaway for a learning project: version from day one, even with no v2 in sight. Starting at v1 costs nothing now and means you have already built the habit of treating the published contract as something you do not casually change, rather than learning that lesson later via a breaking change that upsets real users.

14.2 Rate limiting tiers

Different endpoint categories warrant different limits, reflecting both abuse risk and legitimate usage patterns:

Suggested rate limit tiers

Category	Suggested limit	Reasoning
Auth endpoints (login,	5-10 requests per	These are the highest-value target for credential-

Category	Suggested limit	Reasoning
password reset)	minute per IP	stuffing and brute-force attacks; the limit should be tight enough to make automated guessing impractical while still allowing a real user who mistyped their password a few times.
Mutating financial endpoints (transactions, repayments)	Per-user limit, generous, for example 60 per minute	A legitimate user rarely needs to fire more than a handful of transfers per minute; this tier exists mainly to contain a runaway client bug, such as an accidental retry loop, rather than to stop a determined attacker, who is better addressed by idempotency and fraud detection than by rate limiting alone.
Read endpoints (list and get)	Per-user limit, high, for example 300 per minute	Reads are cheap and a responsive UI may poll or paginate quickly; the limit here exists mainly to catch a misbehaving client polling far faster than any UI legitimately needs to.
Webhook endpoints	Per-provider, matched to that provider's documented retry behavior	These are not really limiting a malicious actor, since the signature check already does that, so much as protecting your own infrastructure from an upstream provider's retry storm during an incident on their end.

14.3 Deprecation, when it eventually happens

When a field, endpoint, or whole version does need to be retired, the standard, considerate way to do it is machine-readable deprecation notices, not just a changelog entry a client developer might never read:

```
HTTP/1.1 200 OK
Deprecation: true
Sunset: Sat, 31 Jan 2027 00:00:00 GMT
Link: <https://docs.yourplatform.com/migration/v1-to-v2>; rel="deprecation"
```

Notes: the Deprecation and Sunset headers are both real, standardized HTTP headers, not a convention invented for this document, that automated tooling and careful client developers can watch for programmatically. A client's CI pipeline can fail a build the moment it notices a Sunset header has appeared on an endpoint it calls, surfacing the problem months before the actual cutoff rather than the day traffic suddenly breaks. The deprecation window itself, the gap between announcing and actually removing, is typically measured in many months for a consumer-facing financial API, precisely because the cost of breaking a client unexpectedly is so much higher here than in a typical SaaS integration.

15. Suggested Build Order & Glossary

This mirrors the staged roadmap from the schema document, but expressed as API surface instead of database tables -- the two roadmaps are meant to be built together, stage by stage, not as two separate sequences.

15.1 Suggested build order

- Stage 1: Section 2 (auth) and Section 3 (user/KYC profile endpoints only, not yet the verification webhook). Goal: register, log in, fetch your own profile.
- Stage 2: Section 4 (accounts) and the core of Section 6 (create transaction, list and get transaction). This is where the idempotency-key handling from Section 1.3 gets implemented for real, for the first time -- treat it as its own mini-milestone, with a test that fires the same request twice and asserts only one transaction exists.
- Stage 3: Section 5 (cards), integrated against a real payment processor's sandbox, plus the payment-processor webhook from Section 10.2 to handle asynchronous settlement.
- Stage 4: Section 7 (loans) end to end -- application, approval, disbursement, which reuses Section 6's transaction logic, schedule, repayment.
- Stage 5: Section 8 (invoices) -- by this point the action-sub-resource pattern of pay and cancel should feel completely familiar, since it is the same shape used for loans and accounts.
- Stage 6: Section 9 (investments and brokerage), built as a queued background job per the schema document's guidance, plus its endpoints and the optional brokerage webhook from Section 10.4.
- Stage 7: Section 11 (admin and compliance) and Section 13's reconciliation endpoints, closing the loop with the schema document's standing reconciliation process.

15.2 What to produce next: an OpenAPI specification

Once this design is stable, the natural next artifact is a machine-readable OpenAPI 3.1 or 3.2 specification -- a single YAML or JSON file that formally describes every endpoint, request and response schema, and error shape in this document in a standardized format that tooling understands. The payoff is substantial for a relatively modest amount of work: client SDKs, request validation middleware, and interactive documentation such as a Swagger UI page can all be generated automatically from one spec file, rather than written and kept in sync by hand across three or four separate places. Writing the OpenAPI spec is also one of the better ways to discover gaps in a design like this one -- formalizing a request schema in JSON Schema terms tends to surface ambiguities, such as whether a postal code is really optional in every country, that prose specifications can quietly paper over.

15.3 Glossary

Key terms used in this document

Term	Meaning
Idempotency key	A unique, client-generated value attached to a mutating request so that retries are safe -- the server returns the original result instead of repeating the side effect. See Section 1.3.
Bearer token	A credential, typically a JWT, sent in the Authorization header that proves the caller's identity for the duration the token remains valid.
Cursor-based pagination	A pagination style using an opaque, server-generated pointer to the last-seen record rather than a page number, avoiding skipped or duplicated rows when the underlying data changes between requests. See Section 1.5.
RFC 9457 / problem+json	The IETF standard this API's error response format follows, giving every error a consistent type, title, status, and detail shape.
Action sub-resource	An endpoint shaped as a verb-like path segment under a resource, such as a freeze action under an account, used for state transitions that need their own validation and audit trail, rather than a generic field update.
Webhook	An HTTP request an external system sends to your API, on its own schedule, to notify you of an event -- the inverse direction of a normal API call.
Signature verification	The authentication method used for webhooks: the sender signs the payload with a shared secret, and the receiver recomputes the signature to confirm the request is genuine and unmodified.
Hyrum's Law	The observation that with enough API consumers, every observable behavior of a system becomes something somebody depends on, whether or not it was ever documented as a guarantee -- the underlying reason API versioning and careful, additive-only evolution matter.
OAuth authorization code flow	The standard pattern, used in Section 9.1, by which a user grants a third-party application limited access to their account on another service, without ever sharing their password with the requesting application.
OpenAPI specification	A standardized, machine-readable format for describing a REST API's endpoints, schemas, and behavior, enabling automatic generation of documentation, client libraries, and validation code.

15.4 Sources consulted

The conventions in this document, particularly the idempotency-key semantics, the RFC 9457 error format, and cursor-based pagination guidance, draw on the following sources reviewed while preparing it:

- Netguru, "API Design Best Practices: REST, Versioning, Security & Documentation" -- on idempotency keys as a first-class design requirement and cursor-based pagination for high-churn data.
- Digital Applied, "REST API Design in 2026: A Full Engineering Reference" -- on RFC 9457 structured errors, versioning as isolation rather than numbering scheme, and machine-readable deprecation headers.
- MDN Web Docs, "Idempotency-Key header" -- the standardized semantics for the Idempotency-Key header, including the 400/409/422 error cases used throughout Section 1.3 and Section 6.

- Tyk API Gateway, "Implementing idempotency protection in API gateways" -- on UK Open Banking's mandatory x-idempotency-key support and per-client key scoping.
- CalibreOS, "API Design Principles: REST, Idempotency, Versioning, and Error Contracts" -- on Hyrum's Law, additive-only API evolution, and the action-sub-resource pattern for state transitions.
- Stripe Engineering Blog, "Designing robust and predictable APIs with idempotency" -- the original reference implementation most other idempotency-key guidance, including this document's, is modeled on.